

ГУАП

КАФЕДРА № 42

ОТЧЕТ
ЗАЩИЩЕН С ОЦЕНКОЙ
ПРЕПОДАВАТЕЛЬ

д-р техн. наук, профессор
должность, уч. степень, звание

подпись, дата

Т. М. Татарникова
инициалы, фамилия

ОТЧЕТ О ЛАБОРАТОРНОЙ РАБОТЕ №2

Моделирование дискретной случайной
величины

по курсу: Моделирование систем

РАБОТУ ВЫПОЛНИЛ

СТУДЕНТ гр. №

4321

подпись, дата

Г.В. Буренков
инициалы, фамилия

Санкт-Петербург 2026

СОДЕРЖАНИЕ

1 Цель работы	2
2 Задание и исходные данные	3
3 Ход выполнения задания	4
3.1 Реализация выборки	4
3.2 Эмпирические оценки и сравнение с теоретическими значениями	5
3.3 Сравнительная оценка гистограмм	6
4 Вывод.....	8
ПРИЛОЖЕНИЕ А	9

1 Цель работы

Программная реализация датчика дискретной случайной величины по заданному закону распределения и проверка соответствия эмпирических характеристик теоретическим значениям.

2 Задание и исходные данные

В работе необходимо реализовать датчик дискретной случайной величины x , которая принимает конечное множество значений с заданными вероятностями. Номер варианта 1 определяет набор значений x_j и вероятностей p_j из таблицы методических указаний. На основе этих данных требуется сформировать накопленные вероятности и использовать их для генерации выборки.

Таблица 1 Исходные данные

№ пп	Параметры	j						
		1	2	3	4	5	6	7
1	x_j	-73,4	-70,7	-51,5	-43,9	13,3	73,0	73,8
	p_j	0,241	0,023	0,166	0,078	0,272	0,192	0,028

3 Ход выполнения задания

Для моделирования дискретной случайной величины использован стандартный подход с разбиением интервала от 0 до 1 на участки, длины которых равны вероятностям p_j . Далее для каждого сгенерированного значения базовой случайной величины выбирается тот интервал, в который оно попадает, и этому интервалу ставится в соответствие конкретное значение x_j .

После реализации датчика была получена выборка объема N равного 500, как требуется в методических указаниях. Для воспроизводимости результатов использовано фиксированное начальное состояние генератора случайных чисел. По полученной выборке вычислены эмпирические оценки математического ожидания и дисперсии, а затем выполнено сравнение с теоретическими значениями, рассчитанными по формулам методички.

На заключительном этапе выполнен анализ распределения вероятностей по значениям x_j . Были построены графики для эмпирических и теоретических вероятностей, а также их визуальное сопоставление. Графики сохранены в папке `plots` и используются как иллюстрации в отчете. По результатам сравнения сделана итоговая оценка качества моделирования дискретной случайной величины.

3.1 Реализация выборки

Алгоритм генерации построен на сопоставлении отрезков интервала от 0 до 1 с вероятностями заданных значений случайной величины. Сначала для всех вероятностей формируются накопленные границы, которые делят интервал на последовательные участки. Далее для каждого шага моделирования генерируется случайное число на интервале от 0 до 1, определяется номер участка, куда это число попало, и в выборку записывается соответствующее значение x_j . Таким образом реализуется прямое воспроизведение дискретного закона распределения.

После генерации всей последовательности из 500 значений выполняется обработка результатов. По выборке определяются частоты появления каждого значения, затем рассчитываются эмпирические оценки математического ожидания и дисперсии, и проводится сравнение с теоретическими характеристиками. На основе этих же частот строятся гистограммы, по которым визуально оценивается, насколько полученное эмпирическое распределение совпадает с заданным теоретическим. На рисунке 1 изображен результат выборки из 30 значений.

13.3, -43.9, 73.0, 13.3, -73.4, 73.8, 13.3, 73.0, -73.4, -43.9, -51.5, 73.0, 13.3, 73.0, -43.9, -73.4, 13.3, -73.4, 73.0, 13.3, 13.3, -51.5, 73.0, 73.0, 13.3, -73.4, -43.9, -73.4, -73.4, 13.3

Рисунок 1 – Выборка из 30 значений, созданная алгоритмом

3.2 Эмпирические оценки и сравнение с теоретическими значениями

Теоретические характеристики вычислены по исходному распределению варианта 1, а эмпирические характеристики определены по сгенерированной выборке объема 500. В результате расчетов получено, что теоретическое математическое ожидание равно -11.588700, теоретическая дисперсия равна 3093.447542, при этом эмпирическое математическое ожидание равно -12.047000, а эмпирическая дисперсия равна 3064.512731. Абсолютные расхождения составили 0.458300 для математического ожидания и 28.934811 для дисперсии.

Анализ полученных данных показывает, что эмпирические оценки близки к теоретическим, а знак и порядок величин полностью совпадают с ожидаемыми значениями для заданного распределения. Наблюдаемая разница является статистически естественной для конечного объема выборки и не указывает на ошибку алгоритма. Следовательно, реализация датчика корректно воспроизводит основные числовые характеристики дискретной случайной величины, заданной в варианте 1. На рисунке 2 изображена итоговая таблица с теоретическими и эмпирическими показателями.

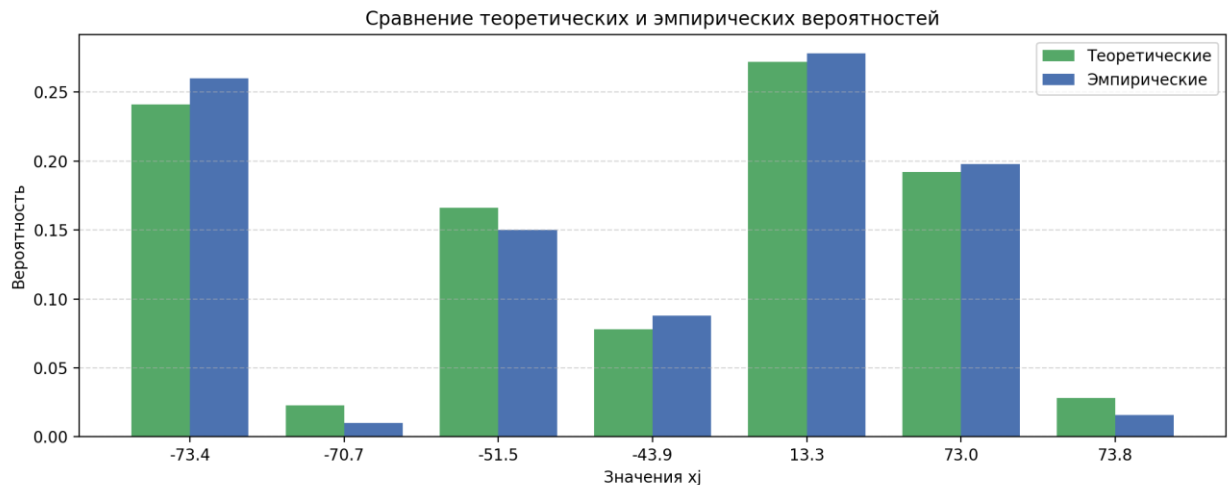


Рисунок 2 – Сравнение теоретических и эмпирических значений математического ожидания и дисперсии

3.3 Сравнительная оценка гистограмм

Сравнение гистограмм показывает, что форма эмпирического распределения в целом повторяет форму теоретического распределения. Наибольшие вероятности соответствуют тем же значениям x_j , что и в исходных данных, а маловероятные значения также встречаются реже. Небольшие различия по высоте отдельных столбцов обусловлены случайным характером выборки и ее конечным объемом.

Дополнительная сравнительная диаграмма подтверждает близость теоретических и эмпирических вероятностей по всем значениям x_j . Следовательно, реализованный алгоритм моделирования корректно воспроизводит заданное распределение дискретной случайной величины для варианта 1. На рисунке 3 изображено прямое сопоставление теоретических и эмпирических вероятностей по каждому значению x_j .

Итоговые результаты ЛР2 (вариант 1)

Показатель	Значение
M(x) теоретическое	-11.588700
D(x) теоретическое	3093.447542
M*(x) эмпирическое	-12.047000
D*(x) эмпирическое	3064.512731
M - M*	0.458300
D - D*	28.934811

Рисунок 3 – Сравнительная диаграмма теоретических и эмпирических вероятностей

4 Вывод

В ходе лабораторной работы реализован датчик дискретной случайной величины по заданным значениям и вероятностям варианта 1. Сгенерирована выборка из 500 значений, что полностью соответствует требованиям задания.

Вычисленные эмпирические оценки математического ожидания и дисперсии близки к теоретическим значениям. Построенные гистограммы показывают согласование эмпирического и теоретического распределений, а выявленные отклонения находятся в пределах статистически ожидаемых для конечного объема наблюдений.

Таким образом, поставленная цель работы достигнута. Разработанная программная реализация корректно моделирует дискретную случайную величину с заданным законом распределения и может применяться в дальнейших задачах имитационного моделирования.

ПРИЛОЖЕНИЕ А

Листинг с кодом программы

В данном разделе представлен исходный код моей части программы:

```
from pathlib import Path

import matplotlib.pyplot as plt
import numpy as np

# Вариант 1 (таблица 3.2 из методички)
X_VALUES = np.array([-73.4, -70.7, -51.5, -43.9, 13.3, 73.0, 73.8], dtype=float)
P_VALUES = np.array([0.241, 0.023, 0.166, 0.078, 0.272, 0.192, 0.028],
dtype=float)
N_SAMPLES = 500
RNG_SEED = 42

def generate_discrete_sample(x: np.ndarray, p: np.ndarray, n: int, rng:
np.random.Generator) -> np.ndarray:
    """Генерация дискретной СВ методом обратной функции распределения."""
    cumulative_prob = np.cumsum(p)
    u = rng.random(n)
    indices = np.searchsorted(cumulative_prob, u, side="right")
    return x[indices]

def compute_theoretical_stats(x: np.ndarray, p: np.ndarray) -> tuple[float, float]:
    m = float(np.sum(x * p))
    d = float(np.sum((x**2) * p) - m**2)
    return m, d
```

```

def compute_empirical_probabilities(x: np.ndarray, sample: np.ndarray) ->
np.ndarray:
    counts = np.array([(sample == xi).sum() for xi in x], dtype=float)
    return counts / sample.size

def save_histograms(x: np.ndarray, p_theoretical: np.ndarray, p_empirical:
np.ndarray, output_dir: Path) -> None:
    output_dir.mkdir(parents=True, exist_ok=True)
    positions = np.arange(len(x))

    fig, axes = plt.subplots(2, 1, figsize=(11, 8), sharex=True)

    axes[0].bar(positions, p_empirical, color="#4C72B0")
    axes[0].set_title("Эмпирическое распределение вероятностей")
    axes[0].set_ylabel("p*")
    axes[0].grid(axis="y", linestyle="--", alpha=0.5)

    axes[1].bar(positions, p_theoretical, color="#55A868")
    axes[1].set_title("Теоретическое распределение вероятностей")
    axes[1].set_ylabel("p")
    axes[1].set_xlabel("Значения xj")
    axes[1].grid(axis="y", linestyle="--", alpha=0.5)
    axes[1].set_xticks(positions)
    axes[1].set_xticklabels([f"{val:.1f}" for val in x], rotation=0)

    fig.tight_layout()
    fig.savefig(output_dir / "hist_compare_variant1.png", dpi=200)
    plt.close(fig)

# Дополнительный сравнительный столбчатый график (удобно для отчета)

```

```

fig2, ax2 = plt.subplots(figsize=(11, 4.5))
width = 0.38
    ax2.bar(positions - width / 2, p_theoretical, width=width,
label="Теоретические", color="#55A868")
    ax2.bar(positions + width / 2, p_empirical, width=width, label="Эмпирические",
color="#4C72B0")
    ax2.set_title("Сравнение теоретических и эмпирических вероятностей")
    ax2.set_xlabel("Значения xj")
    ax2.set_ylabel("Вероятность")
    ax2.set_xticks(positions)
    ax2.set_xticklabels([f"{val:.1f}" for val in x], rotation=0)
    ax2.grid(axis="y", linestyle="--", alpha=0.5)
    ax2.legend()
fig2.tight_layout()
fig2.savefig(output_dir / "bar_compare_variant1.png", dpi=200)
plt.close(fig2)

```

```

def save_results_figure(
    m_theoretical: float,
    d_theoretical: float,
    m_empirical: float,
    d_empirical: float,
    output_dir: Path,
) -> None:

```

```

fig, ax = plt.subplots(figsize=(10, 3.8))
ax.axis("off")
ax.set_title("Итоговые результаты ЛР2 (вариант 1)", pad=12)

```

```

rows = [

```

```

["M(x) теоретическое", f"{m_theoretical:.6f}"],
["D(x) теоретическое", f"{d_theoretical:.6f}"],
["M*(x) эмпирическое", f"{m_empirical:.6f}"],
["D*(x) эмпирическое", f"{d_empirical:.6f}"],
["|M - M*|", f"{abs(m_theoretical - m_empirical):.6f}"],
["|D - D*|", f"{abs(d_theoretical - d_empirical):.6f}"],
]

```

```

table = ax.table(
    cellText=rows,
    colLabels=["Показатель", "Значение"],
    cellLoc="left",
    colLoc="center",
    loc="center",
)
table.auto_set_font_size(False)
table.set_fontsize(11)
table.scale(1, 1.4)

fig.tight_layout()
fig.savefig(output_dir / "results_variant1.png", dpi=220)
plt.close(fig)

```

```

def write_report_files(
    sample: np.ndarray,
    x: np.ndarray,
    p_theoretical: np.ndarray,
    p_empirical: np.ndarray,
    m_theoretical: float,
    d_theoretical: float,

```

```

    m_empirical: float,
    d_empirical: float,
) -> None:
    first_30 = ", ".join(f"{v:.1f}" for v in sample[:30])
    probabilities_table = "\n".join(
        f"| {xj:.1f} | {pt:.3f} | {pe:.3f} |"
        for xj, pt, pe in zip(x, p_theoretical, p_empirical)
    )

def main() -> None:
    if not np.isclose(P_VALUES.sum(), 1.0):
        raise ValueError("Сумма вероятностей pj должна быть равна 1.")

    rng = np.random.default_rng(RNG_SEED)
    sample = generate_discrete_sample(X_VALUES, P_VALUES, N_SAMPLES,
rng)

    m_theoretical, d_theoretical = compute_theoretical_stats(X_VALUES,
P_VALUES)
    m_empirical = float(np.mean(sample))
    d_empirical = float(np.var(sample))
    p_empirical = compute_empirical_probabilities(X_VALUES, sample)

    plots_dir = Path("plots")
    save_histograms(X_VALUES, P_VALUES, p_empirical, plots_dir)
    save_results_figure(m_theoretical, d_theoretical, m_empirical, d_empirical,
plots_dir)
    write_report_files(
        sample,

```

```

X_VALUES,
P_VALUES,
p_empirical,
m_theoretical,
d_theoretical,
m_empirical,
d_empirical,
)

print("ЛР2 (вариант 1) выполнена.")
print(f"N = {N_SAMPLES}, seed = {RNG_SEED}")
print(f"M_theoretical = {m_theoretical:.6f}")
print(f"D_theoretical = {d_theoretical:.6f}")
print(f"M_empirical = {m_empirical:.6f}")
print(f"D_empirical = {d_empirical:.6f}")

    print("Файлы отчета: lab2_report.md, report_lab2_variant1.txt,
first30_variant1.txt")

        print("Графики: plots/hist_compare_variant1.png,
plots/bar_compare_variant1.png, plots/results_variant1.png")

if __name__ == "__main__":
    main()

if __name__ == "__main__":
    main()

```